MVP Backend Technical Specification: Git Repository Performance Optimization Platform

1. MVP Purpose and Scope

The backend for this Minimum Viable Product (MVP) processes Git repository analytics and translates them into actionable performance optimizations within an eight-week implementation scope. It bridges the gap between raw statistics and practical application by automating the identification of performance bottlenecks based on repository characteristics and developer workloads.

The backend achieves this by:

- **Repository Ingestion & Analysis:** Extracting metrics like repository size, file/object counts, commit history, and identifying large objects.
- **Workload Profiling:** Evaluating metrics against specific contexts such as local development, CI/CD, and monorepo development.
- **Recommendation Engine:** Applying predefined rules to suggest Git features like partial clone, shallow clone, sparse checkout, and maintenance options.
- **Benchmarking:** Executing controlled experiments to measure operation time, data transferred, and disk usage for selected optimizations.

2. Database Entities

The logical data model supports the tracking of users, repositories, analysis metrics, optimization rules, and benchmark results.

Entity	Primary Key (PK)	Foreign Keys (FK)	Attributes	Description / Purpose
USER	user_id	None	name, email, password, role	Stores user identity and access data.

WORKLOAD_PROFILE	workload_id	None	name, description	Defines supported operational contexts (e.g., CI/CD, monorepo).
REPOSITORY	repository_id	user_id, workload_id	name, source, url/path	Stores repository references and links to the owning user and classified workload.
ANALYSIS	analysis_id	repository_id	analysis_date, status, duration	Tracks individual analysis runs for a given repository.
METRIC	metric_id	analysis_id	metric_type, metric_value, unit	Stores individual data points (e.g., file counts, repo size) extracted during analysis.
RECOMMENDATION	recommendation_id	None	title, description, category, severity	Defines optimization rules and categories available in the system.

TEMPLATE	template_id	recommendation_id	title, content, format	Stores specific commands or configurations mapped to a recommendation.
BENCHMARK	benchmark_id	analysis_id	operation, configuration, duration, data_transferred, disk_usage	Records results of controlled performance experiments.
ANALYSIS_RECOMMENDATION	analysis_id, recommendation_id	analysis_id, recommendation_id	relevance	A mapping table linking recommendations triggered during an analysis, including a relevance score.

3. Entity Relationships

The database utilizes a relational structure to link user data through to final benchmark reports.

- **USER to REPOSITORY (1:N):** One user can own multiple repositories.
- **WORKLOAD_PROFILE to REPOSITORY (1:N):** One workload profile classifies multiple repositories.
- **REPOSITORY to ANALYSIS (1:N):** One repository can undergo multiple analysis runs over time.
- **ANALYSIS to METRIC (1:N):** One analysis produces multiple extracted metrics.
- **ANALYSIS to RECOMMENDATION (M:N):** Resolved by the ANALYSIS_RECOMMENDATION composite table; one analysis can trigger

multiple recommendations, and a single recommendation rule applies to multiple analyses.

- **RECOMMENDATION to TEMPLATE (1:N):** One recommendation can have multiple configuration templates or formats.
- **ANALYSIS to BENCHMARK (1:N):** One analysis can have multiple benchmarks recorded against it for different operations.

4. Backend API Routes

The routing architecture aligns with the system workflow defined in the system analysis phase: Submit Repository → Select Workload → Analyze Repository → Identify Issues → Generate Recommendations → Apply/Inspect Template → Benchmark → View Results.

Workload & Repository Management

- `GET /api/v1/workloads`
  - Retrieves the list of available workload profiles (Local Development, CI/CD, Monorepo).
- `POST /api/v1/repositories`
  - Accepts repository information (URL/path) and the user's selected workload profile to register the repository in the system.
- `GET /api/v1/repositories/:repository_id`
  - Retrieves repository details and its associated workload profile.

Analysis & Metrics Extraction

- `POST /api/v1/analyses`
  - Triggers the Repository Analysis Module for a specific `repository_id` to extract structure, object, history, and storage metrics.
- `GET /api/v1/analyses/:analysis_id`
  - Retrieves the status, duration, and all associated `METRIC` records generated during the analysis run.

Recommendations & Templates

- `GET /api/v1/analyses/:analysis_id/recommendations`
-
    - Triggers the Recommendation Engine to cross-reference extracted metrics and the workload profile against predefined rules. Returns the identified issues, recommended optimizations, and explanations.
    -
- `GET /api/v1/recommendations/:recommendation_id/templates`
-
    - Retrieves the specific ready-to-use commands or configuration templates associated with a recommended optimization.
    -

Benchmarking & Reporting

- `POST /api/v1/benchmarks`
-
    - Accepts a benchmark request containing an `analysis_id` and a selected optimization configuration. Executes the controlled experiment (e.g., clone performance, working-tree performance).
    -
- `GET /api/v1/analyses/:analysis_id/benchmarks`
-
    - Retrieves the results of all executed benchmarks for an analysis, returning measurable indicators like operation duration, data transferred, and disk usage for dashboard reporting.